Binary File Handling in C language

06 June 2026 20:19

A binary file is a file that stores data in the same format as it is stored in the computer memory(binary form). Unlike text files, binary files are not human-readable because they contain data as a sequence of bytes.

Key points:

- **Data is stored in binary format**
- **Faster and more efficient**
- **Non-human readable.**
- **Examples: .dat, .bin**

Advantages of Binary Files

- **Faster read/write operation**
- **Occupies less storage space**
- **Preserves the exact memory representation of data**
- **Suitable for storing structures and records.**

Opening a Binary File in C:

```
FILE *fp = fopen("path and File name", "wb");
```

Binary File modes

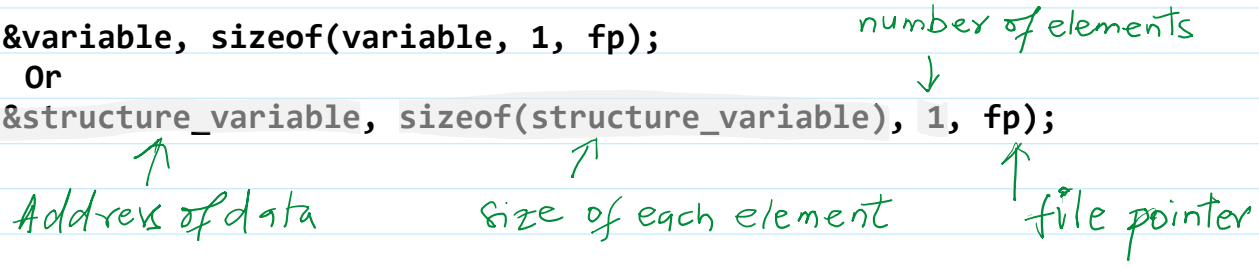
Mode	Description
rb	Read binary file
wb	Write binary file
ab	Append binary file
rb+	Read and write binary file
wb+	Read and write (overwrite)
ab+	Read and append

Important binary File functions

1. fwrite(): Used to write data into a binary file

Syntax:

```
fwrite(&variable, sizeof(variable, 1, fp);
        Or
fwrite(&structure_variable, sizeof(structure_variable), 1, fp);
```



2. fread(): Used to read data from a binary file

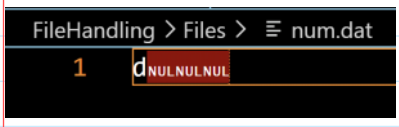
Syntax:

```
fread(&variable, sizeof(variable), 1, fp);
```

Example to write and read from binary file:

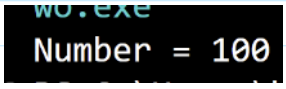
```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    FILE *fp;
    int n = 100;
    fp = fopen("../Files/num.dat", "wb");
    fwrite(&n, sizeof(n), 1, fp);
    fclose(fp);
    return 0;
}
```

The content of binary file:




```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    FILE *fp;
    int n;
    fp = fopen("../Files/num.dat", "rb");
    fread(&n, sizeof(n), 1, fp);
    printf("Number = %d", n);
    fclose(fp);
    return 0;
}
```

Output:



Binary File Handling with Structures

Structures are commonly stored in binary files because the entire record can be written or read in one operation.

Writing Data:

```c
#include<stdio.h>
#include<string.h>
typedef struct Student
{
    /* data */
```

```
    int roll;
    char name[30];
    float marks;
}Stud;
int main(int argc, char const *argv[])
{
    FILE *fp;
    char path[100] = "../Files/";
    char fileName[100];
    printf("Enter file name: ");
    scanf("%s", fileName);
    strcat(path, fileName);

    fp = fopen(path,"wb");
    if(fp == NULL){
        printf("\nFile not created!\n");
        return 1;
    }
    Stud s1 = {101, "Soumodip", 89.6};
    fwrite(&s1, sizeof(s1), 1, fp);
    fclose(fp);
    printf("Record stored successfully.");
    return 0;
}
```

Read Binary File:

```
#include<stdio.h>
#include<string.h>
typedef struct Student
{
    /* data */
    int roll;
    char name[30];
    float marks;
}Stud;
int main(int argc, char const *argv[])
{
    FILE *fp;
    char path[100] = "../Files/";
    char fileName[100];
    printf("Enter file name: ");
    scanf("%s", fileName);
    strcat(path, fileName);

    fp = fopen(path,"rb");
```

```
    if(fp == NULL){
        printf("\nFile not created!\n");
        return 1;
    }
    Stud s1 ;
    fread(&s1, sizeof(s1), 1, fp);
    printf("Roll number = %d\n", s1.roll);
    printf("Name = %s\n", s1.name);
    printf("marks = %.2f\n", s1.marks);
    fclose(fp);

    return 0;
}
```

Output:

```
Enter file name: Student.dat
Roll number = 101
Name = Soumodip
marks = 89.60
```

CRUD Operations on Binary Files

C-> Create
R-> Read
U-> Update
D-> Delete

Create (Insert Record)

```
fwrite(&student, sizeof(student), 1, fp);
```

Read all records

```
while(fread(&s1, sizeof(s1), 1, fp)){
        printf("Roll number = %d\n", s1.roll);
        printf("Name = %s\n", s1.name);
        printf("marks = %.2f\n", s1.marks);
    }
    fclose(fp);
```

Random Access in Binary Files

Random access allows direct access to any record without reading previous records.

fseek(): Move the file pointer.

Syntax:

```
fseek(fp, offset, origin);
```

Origins

Constants	Meaning
SEEK_SET	Beginning of file
SEEK_CUR	Current position
SEEK_END	End of file

Example:

```
fseek(fp, 3*sizeof(student), SEEK_SET);
```

The above statement moves the file pointer on the front of 4th record.

---

ftell(): returns current file position.

```
long pos;

pos = ftell(fp);

printf("Current Position = %d", pos);
```

---

rewind(): moves file pointer to the beginning

```
rewind(fp);
```

Homework:

WAP which is doing the CRUD operations on Student.dat file, write separate function for each operations. Design menu to call these functions as per requirement.